

1a) "Main" representa o endereço de código onde a execução deve começar.

1b) Os rótulos oferecem maior facilidade de compreensão e intuitividade pois permitem que tanto o computador chegue diretamente ao trecho sem necessidade de cálculos quanto melhoram a legibilidade do código, para que o programador não precise saber o que está no trecho de memória específico, ao invés, ele apenas busca pelo mesmo rótulo.

1c) Em resumo, rótulos servem para conveniência humana, e o endereço numérico é a realidade física (o número da célula de memória). É o que o hardware realmente precisa para executar.

1d)

```
.globl main
.text

main:

li t0, 4
ecall

addi a0, a0, 2

li t0, 1
ecall

j main
```

2) e 3)

```
1  .globl main
2  .text
3
4  main:
5
6      li t0, 4
7      ecall
8
9      bne(s0, zero, end)
10     addi a0, a0, 2
11
12     li t0, 1
13     ecall
14     j main
15
16 end:
17     li t0, 10
18     ecall
```

4)

```
RISC-V Assembly
1 .globl main
2 .text
3
4     li t1, 10
5
6 main:
7
8     li t0, 4
9     ecall
10
11    bne a0, zero, end
12    bge a0, t1, end
13    addi a0, a0, 2
14
15    li t0, 1
16    ecall
17    j main
18
19 end:
20     li t0, 10
21     ecall
```

5)

```
4 main:
5
6     li t0, 4
7     ecall
8     bne a0, zero, end
9
10    mv s0, a0    #salva o 1 numero
11
12    li t0, 4
13    ecall
14    bne a0, zero, end
15
16    bge s0, a0, imprim
17
18    li t0, 1     #imprime o outro
19    ecall
20    j main
21
22 imprim:
23     li s0, 1
24     j main
25
26 end:
27     li t0, 10
28     ecall
```

6).data

Define a constante 10 (ASCII 45 para o sinal '-')

MENOS_CHAR: .byte 45

.text

.globl main

main:

1. LEITURA DO NÚMERO (N)

li t0, 4 # Syscall: Ler Inteiro (código 4)

ecall # N lido para a0

**mv s0, a0 # SALVA NÚMERO ORIGINAL (N) em s0
(para usar em comparações)**

**li t1, 10 # Carrega a constante 10 em t1 (para a
comparação e cálculo)**

2. DECISÃO (IF N < 10)

**# Se N (em s0) for MENOR que 10 (em t1), vá para a
ROTA_MENOR.**

blt s0, t1, ROTA_MENOR

--- ROTA MAIOR (N >= 10): Apenas Imprime N ---

**# N já está em s0. Move s0 para a0 para imprimir.
mv a0, s0**

**li t0, 1 # Syscall: Imprimir Inteiro (código 1)
ecall**

j END_PROGRAMA # Termina

ROTA_MENOR:

**# --- ROTA MENOR (N < 10): Calcula (10 - N), Imprime Sinal e
Imprime Resultado ---**

3. CÁLCULO: 10 - N

Subtração em Assembly é feita invertendo a ordem na instrução SUB.

Ex: sub destino, op1, op2 -> destino = op1 - op2

Queremos: s0 = 10 - N

sub s0, t1, s0 # s0 = t1 (10) - s0 (N). O resultado é o valor absoluto do cálculo (10-N).

4. IMPRIME O SINAL DE MENOS (-)

Syscall para Imprimir Caractere (geralmente código 11)

li t0, 11

Carrega o código ASCII do '-' (45) no registrador de argumento a0.

li a0, 45

ecall # Imprime o '-'

5. IMPRIME O RESULTADO (10 - N)

O resultado (10-N) está em s0. Move para a0 para imprimir.

mv a0, s0

li t0, 1 # Syscall: Imprimir Inteiro (código 1)

ecall # Imprime o resultado

A execução cai sequencialmente em END_PROGRAMA

END_PROGRAMA:

Syscall: Exit (código 10)

li t0, 10

ecall